ATIVIDADE ACADÊMICA · ENGENHARIA DE SOFTWARE

Manual de Gestão de Código e Entrega de Software

Sistema DKRT: Auditor de Conformidade de Entidades de Contexto

Estratégia de branches, revisão, CI/CD, hotfix, feature flags e organização das aplicações

Grupo: [preencher com os nomes dos integrantes]

Repositório: TCC-CEFSA-DKRT/TCC_DKRT (GitHub)

Versão 1.0 · outubro de 2026

Sumário

Apresentação e contexto..... 3
1. Estratégia de versionamento..... 4
2. Processo de desenvolvimento..... 6
3. Integração Contínua (CI)..... 8
4. Entrega e implantação (CD)..... 10
5. Tratamento de bugs em produção..... 12
6. Funcionalidades ainda não concluídas..... 13
7. Arquitetura da solução e pipelines..... 14
8. Decisões e justificativas..... 16
9. Estado atual e plano de adoção..... 17
Referências..... 18

Como ler: as seções 1 a 7 respondem, na ordem, às sete perguntas do enunciado. Em cada uma, a caixa **Estado atual** diz o que já existe no repositório hoje e o que é meta; a seção 9 consolida essa diferença num plano de adoção.

Figuras: Figura 1 (workflow Git, seção 1) e Figura 2 (pipeline de CI/CD e hotfix, seção 3).

Apresentação e contexto

Este manual define como o grupo gerencia o código-fonte do sistema DKRT e como cada alteração chega aos usuários. A regra de ouro é simples: **a branch `main` está sempre pronta para ser implantada**, todo código entra nela por Pull Request revisado e validado por automação, e o que ainda não deve aparecer para o usuário entra desligado, atrás de uma feature flag.

O sistema em uma linha

O DKRT audita entidades de um Context Broker (Orion, padrão NGSI-v2) contra os Smart Data Models, em dois níveis: validação estrutural e heurística determinística (Nível 1) e análise semântica por um SLM local com RAG (Nível 2). O código é um monolito modular em arquitetura hexagonal (`domain`, `application`, `infrastructure`, `interfaces`), com um frontend Angular separado.

O que é entregue e onde roda

Componente	Tecnologia	Produção (ADR-0017)	Self-hosted (ADR-0019)
API + Worker	Python 3.11, FastAPI; um único pacote `src/auditor`	3 funções AWS Lambda (api, worker, worker-dlq) de uma mesma imagem de contêiner no ECR, atrás de um ALB	Um processo uvicorn (API + worker) em contêiner (`selfhosted/Dockerfile`)
Frontend	Angular 19	Build estático em S3 + CloudFront	Build estático servido por nginx (`selfhosted/frontend.Dockerfile`)
Banco, fila e RAG	PostgreSQL + pgvector; migrations Alembic; SQS	RDS e SQS	Postgres em contêiner; fila em memória
SLM	Ollama (Qwen2.5)	Instância EC2 com GPU (ADR-0013/0015)	Contêiner Ollama
Infraestrutura	Terraform (`infra/terraform`)	Ambientes `homolog` e `production`	Não se aplica
dataspace-demo	Stack didática com docker compose próprio	Não é implantada	Não se aplica; fornece os 3 cenários de teste de aceitação

Premissas do grupo

- Equipe pequena de estudantes, com todos trabalhando em paralelo no mesmo repositório (privado, organização GitHub do projeto).
- Um produto, uma linha de produção e três ambientes de execução: **local**, **homologação** e **produção** (ADR-0018). O modo self-hosted é um canal de distribuição, não um quarto ambiente do time.
- Processo proporcional ao porte: leve o bastante para ser seguido por todos, rigoroso o bastante para que a `main` nunca quebre.

Resumo das decisões

Pergunta do enunciado	Decisão do grupo
1. Versionamento	**GitHub Flow** com promoção por tags. Branches: `main`, `feature/*` (e `fix/*`, `refactor/*`, `docs/*`, `chore/*`) e `hotfix/*`. Releases marcadas com tags `vMAJOR.MINOR.PATCH`. Não há `develop`, `release/*` nem branch por ambiente.

1. Estratégia de versionamento

1.1 Qual workflow Git foi adotado

O grupo adota o **GitHub Flow**, com uma extensão: as versões que vão a produção são marcadas por **tags semânticas**. Existe uma única branch permanente, a `main`. Todo trabalho acontece em branches curtas que nascem dela e voltam para ela por Pull Request. Homologação e produção não têm branch própria: elas recebem **artefatos** gerados a partir de commits da `main`.

Por que não Git Flow completo (`develop`, `release/*`)? Com poucos integrantes e uma só linha de produção, `develop` e `release/*` acrescentam integrações extras (cada merge duplicado é uma chance de conflito) sem resolver nenhum problema que tenhamos. **Por que não branch por ambiente (`homolog`, `prod`)?** Porque o mesmo código acabaria existindo em versões diferentes em cada branch, e o que foi testado em homologação não seria exatamente o que vai para produção.

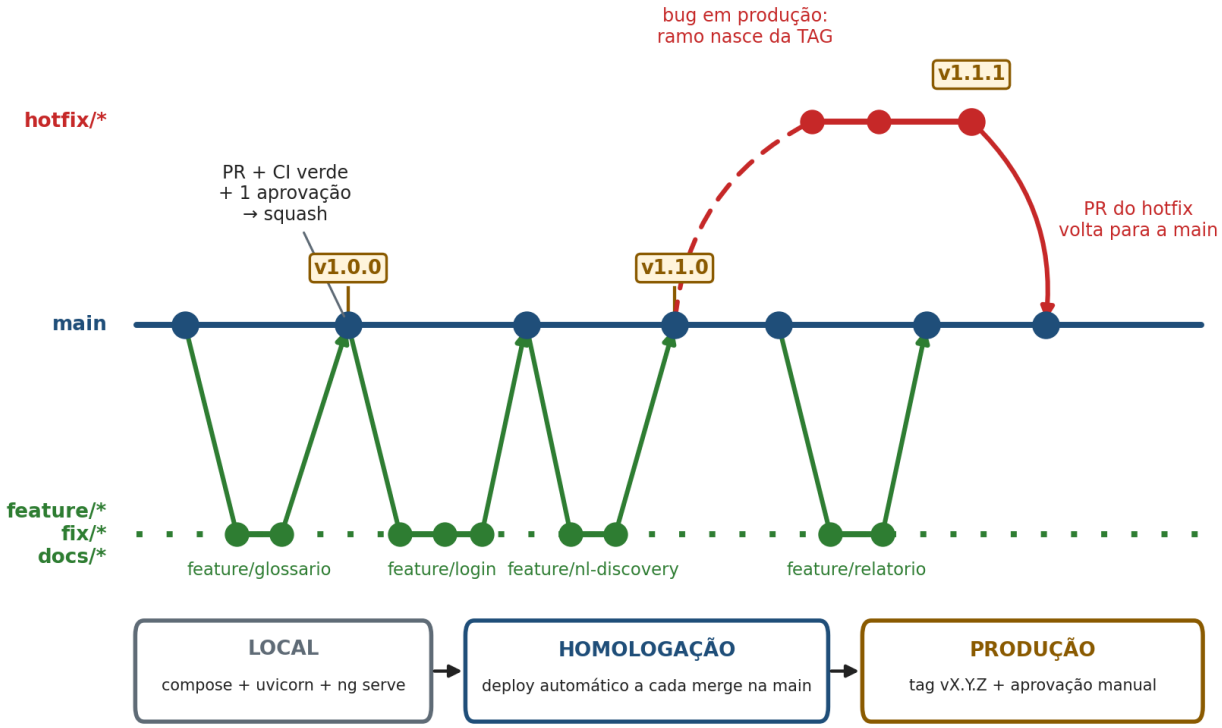


Figura 1: Workflow Git adotado, com os três ambientes à direita da linha do tempo.

1.2 Branches e finalidades

Branch	Finalidade	Origem	Vida útil e encerramento
`main`	Única branch permanente. Contém sempre código que passou no CI e foi revisado. É a base de tudo e a origem de todo build.	Não se aplica	Permanente. Protegida: sem push direto, sem force push, sem exclusão.
`feature/<tema>`	Nova funcionalidade ou evolução. Exemplo: `feature/login-cookie`.	`main` atualizada	Até 5 dias úteis. Encerra no merge do PR e é apagada.
`fix/<tema>`	Correção de bug **não urgente** (segue o fluxo normal).	`main` atualizada	Mesma regra de `feature/*`.

Branch	Finalidade	Origem	Vida útil e encerramento
`refactor/<tema>`, **`docs/<tema>`**, **`chore/<tema>`**	Refatoração sem mudança de comportamento, documentação, tarefas de manutenção (dependências, CI).	`main` atualizada	Mesma regra de `feature/*`.
`hotfix/<descrição>`	Correção **emergencial** de bug em produção.	**A tag** em produção (`vX.Y.Z`)	Até a correção ser publicada e o PR de retorno ser mergeado na `main`; então é apagada.
Tags `vMAJOR.MINOR.PATCH`	Marcam cada versão implantada em produção. Não são branches.	Commit da `main` (ou do `hotfix/*`)	Imutáveis. Nunca são movidas nem apagadas.

1.3 Quando criar e quando encerrar uma branch

Criar quando existir um objetivo pequeno e verificável, de preferência registrado numa issue. A branch sempre sai da `main` recém-atualizada e leva o prefixo do tipo de trabalho. **Encerrar** assim que o PR for mergeado: o GitHub apaga a branch automaticamente (opção *Automatically delete head branches*).

- **Limite de vida:** 5 dias úteis. Trabalho maior que isso é fatiado em PRs menores, com o código incompleto escondido atrás de uma feature flag (seção 6).
- **Branch parada:** sem commits há 7 dias, a branch é revisada em reunião do grupo: retomada, fatiada ou descartada.
- **Mensagens de commit** seguem Conventional Commits (`feat:`, `fix:`, `docs:`, `refactor:`, `test:`, `chore:`). O título do PR vira a mensagem do squash e alimenta as notas de release.
- **Versionamento semântico:** MAJOR para quebra de contrato (API REST ou migração incompatível), MINOR para funcionalidade nova compatível, PATCH para correção. API e frontend compartilham a mesma versão porque são liberados juntos. A primeira tag será `v1.0.0`, correspondente à entrega para a banca.

> **Estado atual no repositório (verificado em 05/10/2026)**
>
> • Existe `main` e branches de trabalho, mas com nomes livres (`improve/refatoracao_simulador_data_space`, `Create-a-glossary-of-synonyms-for-RAG`). O prefixo `improve/` passa a ser equivalente a `refactor/` ou `feature/`; branches antigas não serão renomeadas.
>
> • Não existe nenhuma tag. O `pyproject.toml` declara a versão `0.1.0`.
>
> • O histórico tem commits de merge e merges de `main` dentro de branches de trabalho (PRs #16 a #19). Com squash merge, passa a ser linear.

2. Processo de desenvolvimento

2.1 Como uma funcionalidade é iniciada e integrada

1. **Issue.** A funcionalidade é registrada no GitHub Issues com objetivo e critério de aceite.
2. **Branch.** `git switch main && git pull --ff-only && git switch -c feature/<tema>`.
3. **Desenvolvimento em ciclos pequenos**, com teste escrito junto do código (o projeto já segue TDD). Commits curtos, no padrão Conventional Commits.
4. **Antes de abrir o PR:** rodar os testes localmente (`pytest tests/unit`, `ng test`) e trazer a `main` para dentro da branch, resolvendo conflitos **na branch**, nunca no PR.
5. **Pull Request para a `main`.** O PR é obrigatório para qualquer mudança. Meta de tamanho: até cerca de 400 linhas alteradas. PR em rascunho (*draft*) é bem-vindo para pedir opinião cedo.
6. **CI e revisão** (seções 2.2 e 3). Correções pedidas na revisão entram como novos commits na mesma branch.
7. **Merge.** O Mantenedor faz o **squash merge**: o PR vira um único commit na `main`, com o título no padrão Conventional Commits. A branch é apagada.
8. **Homologação.** O merge dispara a implantação automática em homologação (seção 4).

2.2 Pull Request, aprovação e verificações

Pergunta	Resposta
Haverá Pull Request?	Sim, sempre. Não há push direto na `main`, nem do Mantenedor.
Quem pode aprovar?	Qualquer integrante do grupo **diferente do autor**. Mudanças em `infra/`, `migrations/` e `docs/adr/` exigem também a aprovação do Mantenedor (regra de CODEOWNERS).
Quem pode fazer o merge?	Apenas o **Mantenedor** (Diego de Souza) ou o substituto que o grupo designar em sua ausência. Os demais integrantes têm permissão de escrita para criar branches e abrir PRs, mas não para mergear.
O que é verificado antes do merge?	1) todos os checks do CI verdes (seção 3); 2) pelo menos 1 aprovação válida; 3) branch atualizada com a `main` e sem conflitos; 4) PR descreve a mudança e preenche o checklist do template (testes, migração, flag, ADR); 5) mudança de decisão arquitetural vem acompanhada de ADR em `docs/adr/`; 6) nenhum segredo no diff.

2.3 Por que squash merge e merge só pelo Mantenedor

- **Squash:** cada PR vira um commit, o histórico da `main` fica linear e legível, e reverter uma funcionalidade é reverter um commit. Isso também simplifica o hotfix e o cherry-pick.
- **Mantenedor como porteiro:** com vários integrantes alterando partes diferentes do sistema, um único responsável por apertar o botão garante que ninguém mergeie por engano um PR sem revisão ou com CI vermelho, e dá a quem responde pela entrega o controle do que entra.

2.4 Aplicação das regras na plataforma

As regras acima devem ser impostas pelo GitHub (ruleset da `main`: *Restrict updates* com o Mantenedor na lista de bypass, *Require a pull request*, *Block force pushes*, *Restrict deletions*). **Limite descoberto na prática:** o repositório é privado e a organização está no plano gratuito, e o próprio GitHub avisa que *rulesets* e proteção de branch **não são aplicados** nessa combinação, o que também vale para a exigência de aprovadores em *Environments*. As saídas são: upgrade da organização para o plano Team, usar benefícios acadêmicos do GitHub, ou hospedar o repositório numa conta com plano Pro. Enquanto isso não for decidido, as regras valem **por acordo do grupo**, apoiadas pelo CI e pelo CODEOWNERS.

> **Estado atual no repositório (verificado em 05/10/2026)**
>
> • PRs já são usados (#16 a #19), mas sem regras impostas: qualquer integrante com permissão de escrita consegue mergear.
>
> • Não existem `CODEOWNERS` nem template de PR (a pasta `.github` contém apenas um workflow).
>
> • Evidência do problema que o fluxo resolve: a branch `improve/refatoracao_simulador_data_space` teve o PR #17 mergeado em 23/09, e os 6 commits seguintes (27 a 28/09) ficaram de fora da `main` até o PR #19, em 05/10, exigindo resolução de conflitos.

3. Integração Contínua (CI)

3.1 O que acontece automaticamente após a submissão

O pipeline roda em **todo Pull Request** para a `main` e em todo push na `main`. Além disso, uma execução **agendada semanal** repete as verificações de segurança (novas vulnerabilidades surgem mesmo sem commits) e os testes *golden* do SLM. Cada pasta do monorepo tem seu workflow, acionado por filtro de caminho (seção 7), de modo que uma alteração só no frontend não executa os testes pesados do backend.

Etapa	O que executa	Bloqueia o merge?
Testes do backend	`pytest tests/unit` com cobertura. Em seguida, job de integração com PostgreSQL/pgvector e Orion como serviços, rodando `alembic upgrade head` em banco vazio e `pytest -m integration`.	Sim
Testes do frontend	`npm ci`, `ng test` em Chrome headless e `ng build --configuration production`.	Sim
Qualidade de código	`ruff check` e `mypy`, adotados de forma gradual (primeiro nos arquivos alterados).	Sim, após a adoção
Segurança	`pip-audit` e `npm audit --omit=dev` (vulnerabilidades críticas e altas), varredura de segredos e Dependabot.	Sim (críticas e altas)
Infraestrutura	Quando o PR toca `infra/terraform/**`: `terraform fmt -check`, `validate` e `plan` (o `plan` é publicado para leitura).	`fmt`/`validate`: sim. `plan`: revisão humana
Build das imagens	`docker build` da imagem Lambda (`Dockerfile`) e das imagens self-hosted (`selfhosted/`), sem publicar.	Sim
Testes *golden* do SLM	`pytest -m golden` contra um Ollama real. Pesado demais para cada PR; roda no agendamento e sob demanda.	Não (alerta)

3.2 Quando uma versão está apta para implantação

Uma versão (commit da `main`) só é candidata a produção quando **todas** as condições abaixo forem verdadeiras:

- CI verde: testes unitários e de integração, build do frontend, build das imagens, qualidade e segurança sem pendências críticas.
- As migrações Alembic aplicam em banco vazio **e** sobre o esquema da versão anterior em produção (compatibilidade nos dois sentidos, ver seção 4.4).
- Implantação em homologação concluída, com `GET /health` respondendo e o **teste de aceitação** aprovado: os três cenários do `dataspace-demo` (um conforme, um com erro heurístico, um com erro semântico) devem ser classificados como esperado pela auditoria.
- Nenhum segredo no repositório. Segredos ficam nos Secrets do GitHub e, em produção, no Parameter Store (ADR-0017).
- Notas de release geradas a partir dos títulos dos PRs e tag `vX.Y.Z` criada pelo Mantenedor.

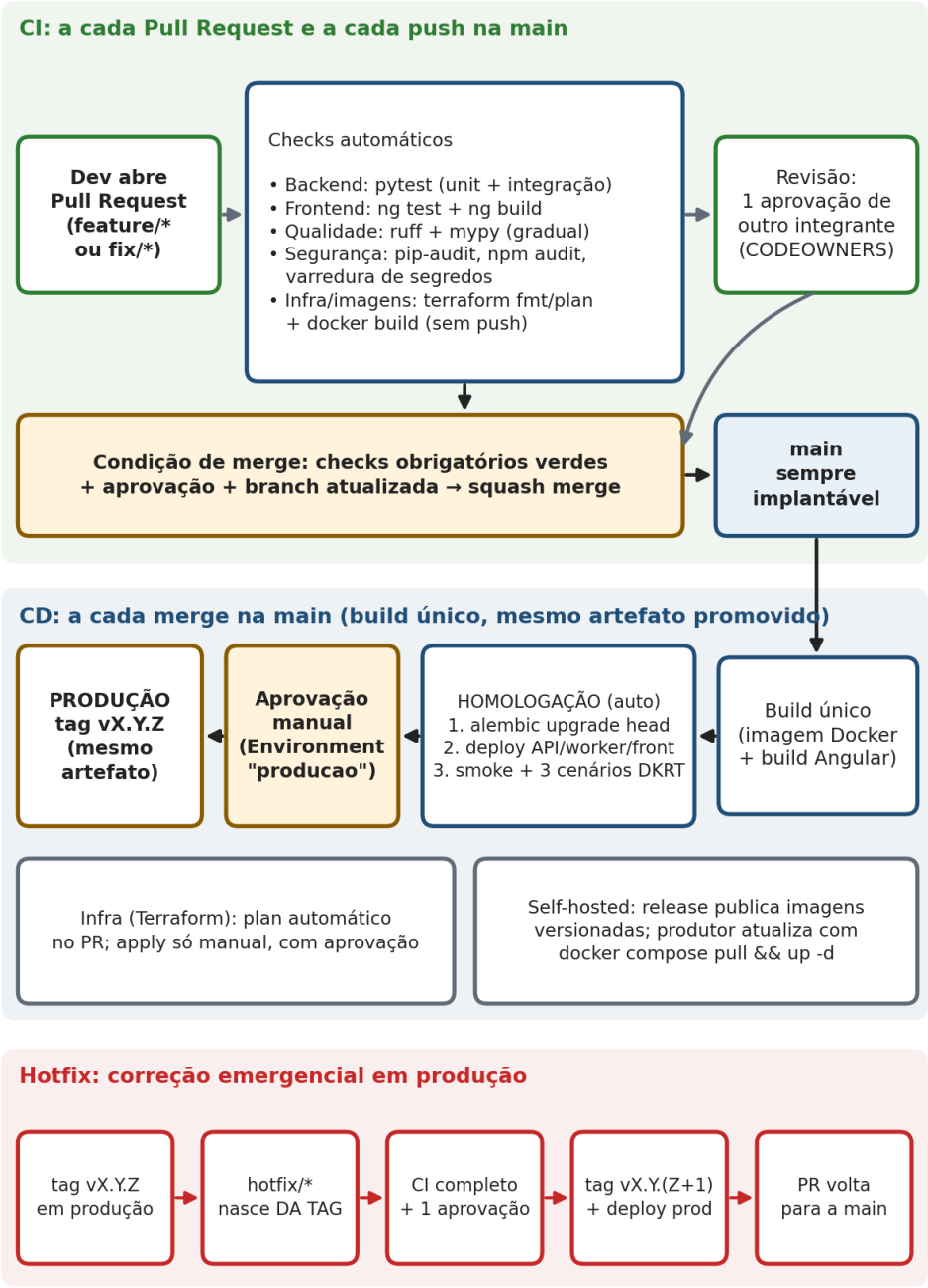


Figura 2: Pipeline de CI/CD e fluxo de hotfix.

Estado atual no repositório (verificado em 05/10/2026)

• Existe um único workflow, `terraform-and-audit.yml`: `terraform plan` em PRs que tocam infraestrutura, `terraform apply` manual com aprovação, e `pip-audit`. O `pip-audit` só é disparado quando o PR toca `infra/` (o filtro de caminho do workflow), então não cobre PRs de código.

• O workflow versionado ainda aponta para `environments/academy`, ambiente aposentado pela ADR-0018. A versão corrigida para `homolog` já está pronta em `Claude outputs/terraform-and-audit-homolog.yml` e falta ser instalada em `.github/workflows/`.

• Os testes existem (56 arquivos em `tests/unit`, 11 em `tests/integration`, 1 em `tests/golden`, com marcadores `integration` e `golden`) e o frontend tem `ng test`, mas **nenhum deles roda em CI hoje**. Não há configuração de `ruff` nem de `mypy` (débito já registrado no próprio workflow).

4. Entrega e implantação (CD)

4.1 Como o software chega a cada ambiente

Princípio central: **construir uma vez, promover o mesmo artefato.** A cada merge na `main`, o pipeline gera uma imagem Docker versionada pelo hash do commit e o build estático do Angular. O que foi testado em homologação é exatamente o que vai para produção, sem recompilar.

Ambiente	Para que serve	Gatilho	Automático?	Aprovação
Local	Desenvolvimento e testes de cada integrante: `docker compose up -d` (Orion, Postgres), `uvicorn` e `ng serve`.	Manual, por quem desenvolve	Não se aplica	Não
Homologação (nuvem, ADR-0018)	Validar a versão antes da banca, com o stack completo e o SLM.	Todo merge na `main`	Sim: migrações, deploy, teste de fumaça e aceitação	Não
Produção (ADR-0017)	Versão mostrada como produto final.	Tag `vX.Y.Z` criada pelo Mantenedor sobre um commit já validado em homologação	Sim, depois da aprovação	**Sim**, manual
Self-hosted (ADR-0019)	Distribuição para o produtor instalar na própria infraestrutura.	Publicação da release	Imagens versionadas publicadas no registry (GHCR)	Não. O produtor atualiza com `docker compose pull && up -d`

4.2 O que é automático e o que é manual

Automático	Manual (com responsável)
Build e testes Publicação da imagem e do build do frontend Migrações Alembic em homologação Deploy em homologação Teste de fumaça (`/health`) e de aceitação (3 cenários) Publicação do frontend (S3 e invalidação do CloudFront) e deploy das funções Lambda em produção, após a aprovação Verificação de saúde após o deploy	**Aprovar a implantação em produção** (Mantenedor ou substituto) **Criar a tag e a release** (Mantenedor) **`terraform apply`** (disparo manual, em ambiente com aprovação: `workflow_dispatch`) Trocar o plano da organização GitHub Publicação de release self-hosted para clientes

4.3 Quem autoriza uma implantação em produção

O ambiente `producao` do GitHub Environments exige aprovação de **um autorizador**: o Mantenedor (Diego de Souza) ou o substituto designado pelo grupo. A opção *Prevent self-review* impede que quem disparou a release aprove a própria implantação quando houver outro autorizador disponível. Nenhum outro papel consegue promover um artefato para produção.

4.4 Banco de dados, rollback e infraestrutura

- **Migrações:** o `alembic upgrade head` roda antes de o novo código receber tráfego. Como código antigo e novo podem coexistir por instantes, as migrações seguem a regra *expandir e contrair*: primeiro adicionam (colunas anuláveis, tabelas novas), e só uma versão depois removem o que ficou sem uso. Isso evita depender de *downgrade*.
- **Rollback:** reimplantar a tag anterior, já que os artefatos são imutáveis (versão e alias das Lambdas, build anterior do frontend com nova invalidação do CloudFront). Como as migrações são compatíveis, voltar o código não exige voltar o banco.
- **Infraestrutura (Terraform):** `plan` automático no PR, `apply` sempre manual. A infraestrutura muda raramente e um `apply` errado derruba o ambiente inteiro, então a aprovação humana vale o atrito.

> **Estado atual no repositório (verificado em 05/10/2026)**
>
> • A infraestrutura de produção (Lambda, ALB, CloudFront/S3, Parameter Store, VPC endpoints) e de homologação (EC2) está descrita em Terraform, em `environments/production` e `environments/homolog`.
>
> • **Não há automação versionada** de build e push da imagem, nem de publicação do frontend ou atualização das Lambdas: hoje o repositório só automatiza Terraform. Esta é a principal lacuna de CD (seção 9).
>
> • Pela ADR-0018 (15/09/2026), a conta AWS de homologação ainda não existia. O Terraform já está pronto para quando ela existir.
>
> • O ambiente de GitHub com aprovação manual já é usado no `terraform apply` (`academy-apply`, a ser renomeado para `homolog-apply`).

5. Tratamento de bugs em produção

5.1 Quando usar hotfix

Hotfix é para falha que afeta a versão em produção e **não pode esperar** o ciclo normal: sistema indisponível, auditoria retornando resultado errado, falha de segurança. Qualquer outro bug segue o fluxo comum, com uma branch `fix/*`.

5.2 Passo a passo

1. **Registrar:** abrir uma issue com a severidade, o impacto e como reproduzir, e avisar o grupo.
2. **Criar a branch a partir da tag em produção:** `git switch -c hotfix/<descricao> vX.Y.Z`. Usar a tag, e não a `main`, é o ponto-chave: a `main` pode já conter funcionalidades que ainda não foram liberadas, e elas não podem ir para produção junto com uma correção emergencial.
3. **Corrigir com um teste que reproduz o bug** antes da correção. Mudança mínima, sem aproveitar para refatorar.
4. **Validar:** abrir o PR e passar pelo CI completo, com 1 aprovação de outro integrante. A aprovação é obrigatória mesmo na urgência.
5. **Publicar:** o Mantenedor cria a tag `vX.Y.(Z+1)` sobre o commit do hotfix. O pipeline gera o artefato, executa o deploy de homologação em paralelo para o teste de fumaça e promove para produção após a aprovação do autorizador.
6. **Devolver à `main`:** em seguida, abrir um PR do `hotfix/*` para a `main` (ou aplicar o `cherry-pick` do commit), resolvendo conflitos nesse PR. Assim a correção nunca se perde na próxima release.
7. **Encerrar:** apagar a branch, avisar o grupo para que as branches em andamento incorporem a `main` atualizada e registrar na issue a causa raiz e o teste de regressão adicionado.

5.3 Como a correção volta para as demais linhas

Como existe uma única linha permanente, a `main`, o retorno é um só: o PR do hotfix. Branches de funcionalidade em andamento recebem a correção na próxima vez que trouxerem a `main` para dentro delas, o que o processo exige antes de abrir o PR (seção 2.1, passo 4). Se a `main` ainda estiver idêntica à tag em produção (nenhum commit novo), o hotfix pode nascer direto da `main` com o mesmo resultado.

> **Estado atual no repositório (verificado em 05/10/2026)**
>
> • Ainda não há tags nem pipeline de produção por tag, portanto o fluxo de hotfix descrito aqui é a meta. Até lá, uma correção emergencial seguiria o fluxo normal de PR com prioridade máxima.

6. Funcionalidades ainda não concluídas

6.1 Decisão: Feature Flag, e não branch de longa duração

Código ainda incompleto **entra na `main` desligado**, protegido por uma feature flag. O grupo não mantém uma branch longa para guardá-lo. A regra vale para tudo que o usuário veria: uma tela, um endpoint, uma etapa do pipeline de auditoria.

6.2 Justificativa

- **Evidência do próprio projeto.** A branch `improve/refatoracao_simulador_data_space` viveu de 22/09 a 05/10. O PR #17 foi mergeado em 23/09 e 6 commits posteriores ficaram 8 dias fora da `main`, até o PR #19, que ainda exigiu resolver conflitos. Branch longa significa integração tardia, conflitos grandes e trabalho que existe só na máquina de quem escreveu.
- **Coerência com o CI/CD.** O princípio de que a `main` está sempre implantável só se sustenta se o trabalho entrar cedo e em fatias pequenas. A flag é o que permite entrar cedo sem expor nada.
- **O projeto já funciona assim.** O sistema usa flags de configuração via `Settings`: `DEPLOYMENT_MODE` (`saas` ou `selfhosted`) e `QUEUE_BACKEND`. E o endpoint `/session` entrega configuração em runtime ao frontend (ADR-0020), então a UI pode reagir a flags sem recompilar.
- **Custo proporcional.** Um serviço externo de flags (LaunchDarkly e similares) seria desproporcional ao porte do projeto. Variáveis em `Settings` custam quase nada e já têm o fluxo de implantação pronto (Parameter Store em produção, `.env` no self-hosted).

6.3 Convenção

Regra	Detalhe
Nome e padrão	`FEATURE_<NOME>` em `Settings` (pydantic-settings), booleano, **desligado por padrão**.
Onde é ligada	Local: `.env`. Homologação: ligada para validar. Produção: Parameter Store, só ligada quando a funcionalidade estiver completa e validada. Self-hosted: `.env`.
Frontend	Lê as flags ativas pelo `/session` (campo `features`) e esconde a rota ou o componente quando desligada.
Testes	Cobrir os dois estados (ligada e desligada). A flag nunca substitui autorização: ela esconde funcionalidade, não protege dado.
Remoção	Depois que a funcionalidade é ligada em produção, a flag é removida em até 2 releases (issue de limpeza criada junto). Flags esquecidas viram dívida.
Exemplo	A intenção *discovery* da tela Natural Language (listar os tipos de entidade do broker) poderia ter entrado na `main` com `FEATURE_NL_DISCOVERY=false`, sendo validada em homologação antes de ser ligada em produção.

Exceção: *spikes* descartáveis (experimentos que não serão entregues) podem usar uma branch própria, com vida de até 5 dias, e são apagados ao final, nunca mergeados.

> **Estado atual no repositório (verificado em 05/10/2026)**
>
> • As flags de configuração mencionadas existem e funcionam. A convenção `FEATURE_*` e o campo `features` no `/session` são a formalização proposta, ainda não implementada.

7. Arquitetura da solução e pipelines

7.1 Mesma estratégia para tudo?

Sim. O projeto é um **monorepo**: backend, frontend, infraestrutura, bundle self-hosted e demonstração didática vivem no mesmo repositório, com uma só `main`, o mesmo fluxo de branches, o mesmo PR e o mesmo critério de aprovação. A arquitetura do código é um **monolito modular** (hexagonal), não microsserviços: há um único bounded context (auditoria de conformidade), um único banco e um único pacote Python que sobe como API ou como worker conforme o ponto de entrada.

Justificativa: para uma equipe pequena, cada regra extra de processo é custo de coordenação. Repositórios separados exigiriam sincronizar versões entre eles, e aqui o contrato entre API e frontend muda junto (os DTOs do frontend espelham `schemas.py` do backend), o que é muito mais simples de revisar num mesmo PR.

7.2 Pipelines independentes por pasta

Mesma estratégia de branches, mas **pipelines independentes**, acionados por filtro de caminho (`paths:`), técnica que o workflow de infraestrutura já usa. Assim cada mudança só executa o que lhe diz respeito.

Componente	Pipeline	Dispara com mudança em	Artefato	Implantável isoladamente?
API + Worker	`backend`	`src/**`, `migrations/**`, `tests/**`, `requirements.txt`, `Dockerfile`	Imagem de contêiner (ECR)	Sim. As três funções (api, worker, worker-dlq) sobem juntas, por design, pois usam a mesma imagem (ADR-0017)
Frontend	`frontend`	`frontend/**`	Build estático (S3/CloudFront)	Sim, desde que o contrato da API seja compatível
Infraestrutura	`infra`	`infra/terraform/**`	Plano Terraform	Sim, com `apply` manual
Bundle self-hosted	`selfhosted`	`selfhosted/**` e as de backend/frontend	Imagens no GHCR	Sim, publicado só em release
dataspace-demo	`demo`	`dataspace-demo/**`	Nenhum (não é implantado)	Não se aplica. O CI roda `test_seed_dkrt.py`; seus cenários servem de massa de teste de aceitação

7.3 Versionamento e componentes implantados separadamente

- **Uma versão de produto** (`vX.Y.Z`) cobre API e frontend, que são liberados juntos. A infraestrutura é rastreada por commit e `plan`, sem tag própria. O `dataspace-demo` é versionado junto, por ser didático.
- **Contrato API e frontend:** mudança incompatível é feita em duas etapas (expandir, depois contrair) ou vira versão MAJOR. Com isso o frontend pode ser publicado sem a API e vice-versa.
- **SLM (Ollama):** não tem build próprio; usa a imagem pública e o modelo definidos no Terraform. Trocar o modelo é uma mudança de infraestrutura, aprovada como tal, e os testes *golden* detectam regressão de qualidade.

7.4 Quando isso deveria mudar

A decisão por monorepo e monolito modular não é definitiva. O grupo reavalia a separação em repositórios ou serviços quando **dois componentes passarem a precisar de ciclos de entrega diferentes na prática**. O

candidato mais provável é o worker de auditoria, que já é um ponto de entrada distinto (`interfaces/worker`, função Lambda própria). A arquitetura hexagonal mantém essa extração barata, e por isso não pagamos o custo de microsserviços antes de precisar.

> **Estado atual no repositório (verificado em 05/10/2026)**
>
> • O repositório já é um monorepo com `src/`, `frontend/`, `infra/`, `selfhosted/` e `dataspace-demo/`, e o workflow existente já usa filtro de caminho para a infraestrutura. Os demais pipelines por pasta ainda não existem.

8. Decisões e justificativas

Resumo das principais escolhas, com a alternativa descartada e o preço aceito. O enunciado pede coerência entre arquitetura, time, branches, integração e entrega: cada linha abaixo se apoia nas premissas da seção de contexto (equipe pequena, um produto, uma linha de produção).

Decisão	Alternativa descartada	Por que	Preço aceito
GitHub Flow com tags	Git Flow (`develop`, `release/*`) e branch por ambiente	Menos integrações, `main` sempre implantável e artefato único testado em todos os ambientes	Exige disciplina de CI e de feature flags
Squash merge	Merge commit ou rebase	Histórico linear, revert de uma funcionalidade = um commit, cherry-pick de hotfix simples	Perde-se o detalhe dos commits intermediários na `main` (ficam no PR)
Merge só pelo Mantenedor	Qualquer integrante mergeia após aprovação	Um responsável pela entrega e menos risco de merge sem revisão	Gargalo no Mantenedor; mitigado pelo substituto designado
Feature flag	Branch longa até a funcionalidade ficar pronta	Integração cedo, sem conflitos grandes (caso real do PR #19); já é o padrão do projeto	Flags precisam ser removidas; dobram os estados a testar
Flags em `Settings`	Serviço externo de feature flags	Custo zero e já integrado a Parameter Store e `.env`	Sem painel nem segmentação por usuário
Build único, artefato promovido	Recompilar para cada ambiente	O que foi testado em homologação é o que vai para produção	Configuração por ambiente fica fora da imagem (variáveis e Parameter Store)
Hotfix a partir da tag	Hotfix a partir da `main`	A `main` pode ter funcionalidades ainda não liberadas	Exige um PR extra de retorno para a `main`
Aprovação manual só em produção	Aprovação em todos os ambientes	Homologação é para errar barato e rápido; produção é o que a banca vê	Produção depende da disponibilidade do autorizador
Monorepo, pipelines por pasta	Multi-repositório ou microsserviços	Contrato API-frontend muda junto; um bounded context; equipe pequena	CI precisa de filtros de caminho bem mantidos
Terraform `apply` manual	Apply automático na `main`	Mudança rara e de alto impacto; hoje já é assim no workflow	Mais lento para infraestrutura

9. Estado atual e plano de adoção

Este manual descreve o processo-alvo. Para ser honesto sobre o ponto de partida, a tabela abaixo compara cada prática com o que **existe hoje no repositório** (verificado em 05/10/2026).

Prática	Hoje	Meta	Etapa
Proteção da `main`	Não imposta (repositório privado em organização no plano gratuito)	Ruleset ativo: PR obrigatório, sem force push, merge restrito ao Mantenedor	1
PR, template e CODEOWNERS	PRs usados (#16 a #19); sem template nem CODEOWNERS	Template com checklist; CODEOWNERS para `infra/`, `migrations/`, `docs/adr/`	1
CI de testes	Inexistente (só Terraform e `pip-audit`)	Pipelines `backend` e `frontend` bloqueando o merge	1
Workflow Terraform	Aponta para `environments/academy` (aposentado, ADR-0018)	Instalar a versão `homolog` já preparada	1
Segurança em CI	`pip-audit` só roda em PRs que tocam `infra/`	`pip-audit` e `npm audit` em todo PR, mais execução semanal	1
Qualidade de código	Sem configuração de `ruff` nem `mypy`	Adoção gradual com `ruff` e `mypy`	2
Convenção de branches e commits	Nomes livres (`improve/*`)	`feature/*`, `fix/*`, `hotfix/*` e Conventional Commits	1
Squash merge	Merge commits e merges de `main` nas branches	Squash merge como único método permitido	1
Tags e releases	Nenhuma tag; versão `0.1.0`	Tag `v1.0.0` na entrega; notas de release automáticas	2
CD de homologação	Sem automação de build/push/deploy; conta AWS pendente (ADR-0018)	Deploy automático a cada merge, com teste de fumaça e aceitação	2
CD de produção	Apenas Terraform está automatizado	Promoção por tag com aprovação no Environment `producao`	3
Feature flags	Flags de configuração existem (`DEPLOYMENT_MODE`, `QUEUE_BACKEND`)	Convenção `FEATURE_*` e campo `features` em `/session`	3
Hotfix e rollback	Sem fluxo formal	Fluxo de hotfix a partir da tag e rollback testado em homologação	3

Etapa 1 (fundação): proteger a `main`, criar template e CODEOWNERS, ligar o CI de testes e corrigir o workflow de Terraform. **Etapa 2 (entrega):** qualidade de código, tags e deploy automático em homologação. **Etapa 3 (maturidade):** produção por tag com aprovação, feature flags formalizadas e ensaio de hotfix e rollback.

Referências

- GitHub Flow: https://docs.github.com/en/get-started/using-github/github-flow
- Conventional Commits: https://www.conventionalcommits.org
- Versionamento Semântico: https://semver.org/lang/pt-BR/
- Feature Toggles (Pete Hodgson): https://martinfowler.com/articles/feature-toggles.html
- HUMBLE, J.; FARLEY, D. Continuous Delivery. Addison-Wesley, 2010.
- Documentos do próprio projeto: ADR-0017 (produção em Lambda), ADR-0018 (homologação), ADR-0019 (modos SaaS e self-hosted), ADR-0020 (sessão e configuração em runtime), em `docs/adr/`.